EPFC

PRO
Programmation orientée objet

Leçon 6

Introduction au concept d'héritage

Les interfaces

Quelques interfaces de la librairie Java

Retour sur la comparaison d'objets

EPFC

Introduction au concept d'héritage

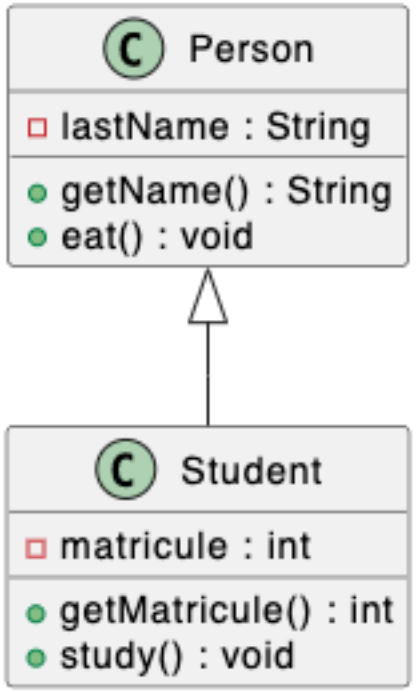
Héritage : définition

L'**héritage** est un mécanisme de programmation orientée objet qui permet à un type d'**hériter des membres d'un autre type**.

Cela favorise la réutilisation du code et la création de hiérarchies de types, où le type dérivé peut **ajouter ou modifier le comportement** du type de base tout en héritant de ses caractéristiques fondamentales.

Dans l'exemple à droite, on peut dire qu'un étudiant (`Student`) est une (*verbe « **être** », relation « **IS A** »*) personne (`Person`).

En UML, cela se matérialise par un triangle fermé au bout d'une relation. La relation est en trait continu dans le cas d'extension et en trait pointillé en cas d'implémentation (voir plus loin).



Héritage : exemple

```java
public class Person {
    private String lastName;

    public String getName() {
        return lastName;
    }

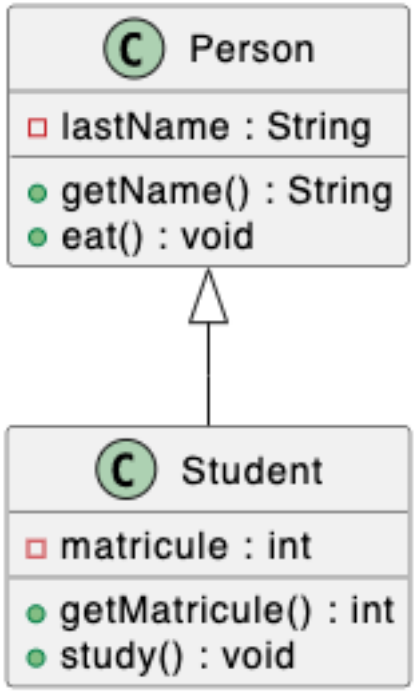
    public void eat() {
        //...
    }
}
```

```java
public class Student extends Person {
    private int matricule;

    public int getMatricule() {
        return matricule;
    }

    public void study() {
        //...
    }
}
```

La classe Student étend (extends) la classe Person




```java
Person p1 = new Person();          ✔
p1.getName();                      ✔
p1.eat();                          ✔
p1.study();                        ✖

Student s1 = new Student();        ✔
s1.getName();                      ✔
s1.eat();                          ✔
s1.study();                        ✔

Person p2 = new Student();         ✔
p2.getName();                      ✔
p2.eat();                          ✔
p2.study();                        ✖

Student s2 = new Person();         ✖
```

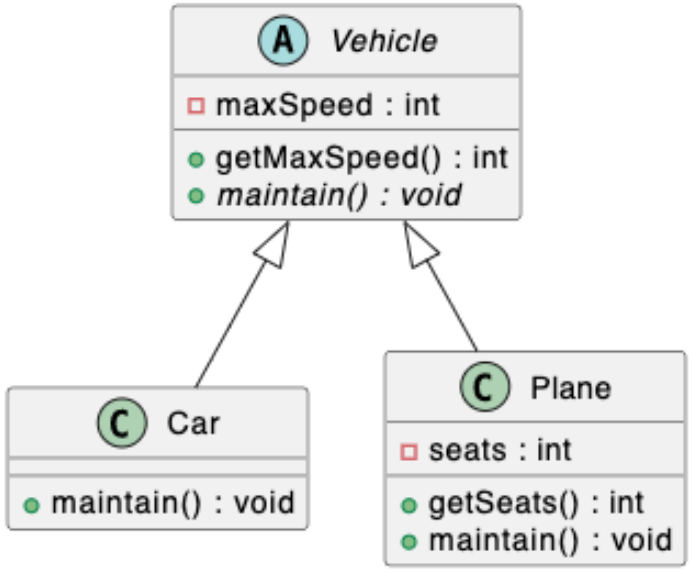
Héritage : exemple de classe abstraite

Une classe peut être **abstraite** (`abstract`).

Elle ne peut **pas être instanciée**.

Comme une classe, elle peut posséder des attributs et des méthodes concrètes. Elle peut en plus posséder des prototypes de méthodes appelés méthodes abstraites.

Si une classe concrète hérite d'une classe abstraite, elle a le devoir de définir toutes les méthodes laissées abstraites.




```java
public abstract class Vehicle {
    private int maxSpeed;

    public int getMaxSpeed() {
        return maxSpeed;
    }

    public abstract void maintain();
}
```

```java
public class Plane extends Vehicle {
    private int seats;

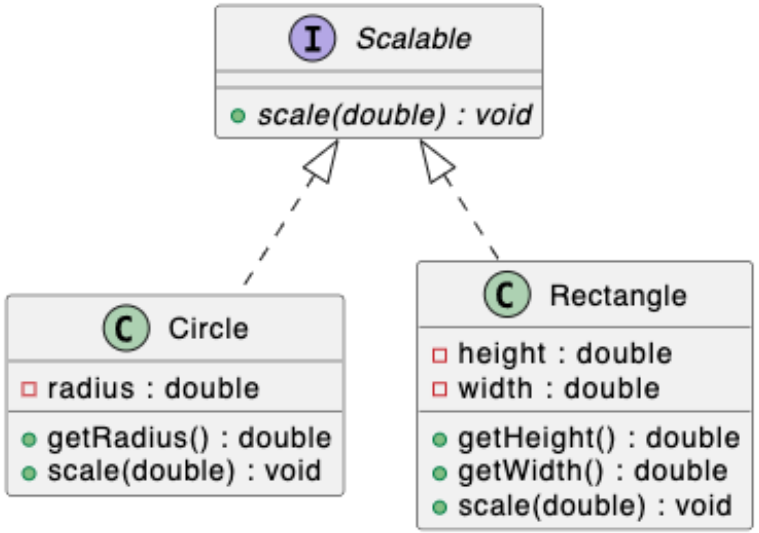
    public int getSeats() {
        return seats;
    }

    @Override
    public void maintain() {
        //...
    }
}
```

```java
~~Vehicle v1 = new Vehicle();~~ ✖
Vehicle v2 = new Car();         ✔
Car c1 = new Car();             ✔
Vehicle v3 = new Plane();       ✔
~~Car c2 = new Plane();~~       ✖
```

```java
public class Car extends Vehicle {
    @Override
    public void maintain() {
        //...
    }
}
```

Héritage : exemple d'interface



Une **interface** est un type encore **plus abstrait** qu'une classe abstraite. Elle ne peut **pas être instanciée ni contenir d'attribut**.

Une classe peut hériter d'une ou plusieurs interface(s). On parle alors d'**implémentation**, `implements`.

En général, on y trouve principalement des méthodes abstraites (voir plus loin).

```java
public interface Scalable {
    void scale(double scaleFactor);
}
```

```java
public class Rectangle implements Scalable {
    private double height;
    private double width;

    public double getWidth() {
        return width;
    }

    public double getHeight() {
        return height;
    }

    @Override
    public void scale(double scaleFactor) {
        height *= scaleFactor;
        width *= scaleFactor;
    }
}
```

```java
public class Circle implements Scalable {
    private double radius;

    public double getRadius() {
        return radius;
    }

    @Override
    public void scale(double scaleFactor) {
        radius *= scaleFactor;
    }
}
```

Les interfaces

Une interface est un contrat

Une interface est un type référence qui peut contenir uniquement :

- des signatures de méthodes (abstraites),
- des méthodes (appelées méthodes par défaut),
- des méthodes statiques
- et des constantes.

Une interface ne peut donc pas être instanciée puisque certaines de ses méthodes sont abstraites (ne contiennent pas de code).

Une classe peut implémenter une ou plusieurs interfaces. Dans ce cas, l'interface définit un contrat vis à vis de cette classe : celle-ci doit implémenter les méthodes abstraites déclarées dans l'interface.

En Java, cela se traduit à l'aide du mot clé `implements`.

Exemple

`Circle` et `Rectangle` implémentent `Scalable`.

Le contrat est de définir la méthode `public void scale(double)`.

```java
public interface Scalable {
    void scale(double scaleFactor);
}
```

```java
public class Circle implements Scalable {
    @Override
    public void scale(double scaleFactor) {
        //...
    }
}
```

```java
public class Rectangle implements Scalable {
    @Override
    public void scale(double scaleFactor) {
        //...
    }
}
```

Un `Rectangle` est un `Scalable`

Une interface, comme une classe, est un type.

Comme `Rectangle` implémente `Scalable`, on dit qu'un `Rectangle` est un `Scalable`.

On peut donc référencer un `Rectangle` avec une variable de type `Scalable`.

```
Rectangle r = new Rectangle();
r.scale(2);

Scalable s = r;
s.scale(1.5);

Rectangle r2 = s;
```

Par contre, l'inverse ne fonctionne pas : un `Scalable` n'est pas un `Rectangle`.

Référents et membres

On ne peut accéder qu'aux membres définis par le type d'un référent.

Dans l'exemple suivant :

- `Scalable` déclare la seule méthode `scale()`
- `Rectangle` est un `Scalable`
- `Rectangle` définit une méthode `getWidth()` et `scale()`

```
Rectangle r = new Rectangle();
r.scale(2);
System.out.println(r.getWith());

Scalable s = r;
s.scale(1.5);
System.out.println(s.getWith());
```

On ne peut donc pas appeler `getWidth()` sur un `Scalable` puisqu'un `Scalable` ne déclare que la méthode `scale()`.

Quelques précisions

Une méthode déclarée dans une interface doit être publique.

Un "attribut" défini dans une interface doit être `public`, `final` et `static` (ce n'est donc pas un attribut, c'est une constante).

Formes équivalentes :

```
public interface MyInterface {
    int MY_CONST = 42;
    int myMethod(double a);
}
```

```
public interface MyInterface {
    public static final int MY_CONST = 42;
    public abstract int myMethod(double a);
}
```

Héritage multiple d'interfaces

Les classes et les interfaces peuvent hériter de plusieurs interfaces.

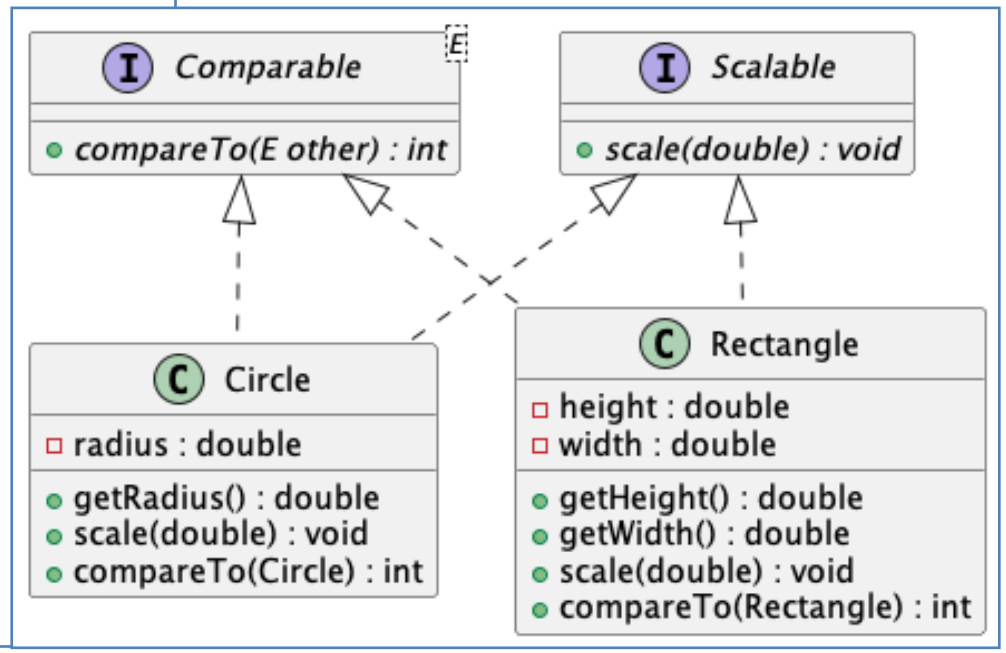
Dans le cas d'une classe qui hérite d'un interface on parlera d'implémentation de l'interface (`implements`).
Dans le cas d'une interface qui hérite d'une interface, on parlera d'extension (`extends`) : voir plus loin.

```
public class Circle implements Scalable, Comparable<Circle> {

    public double radius;

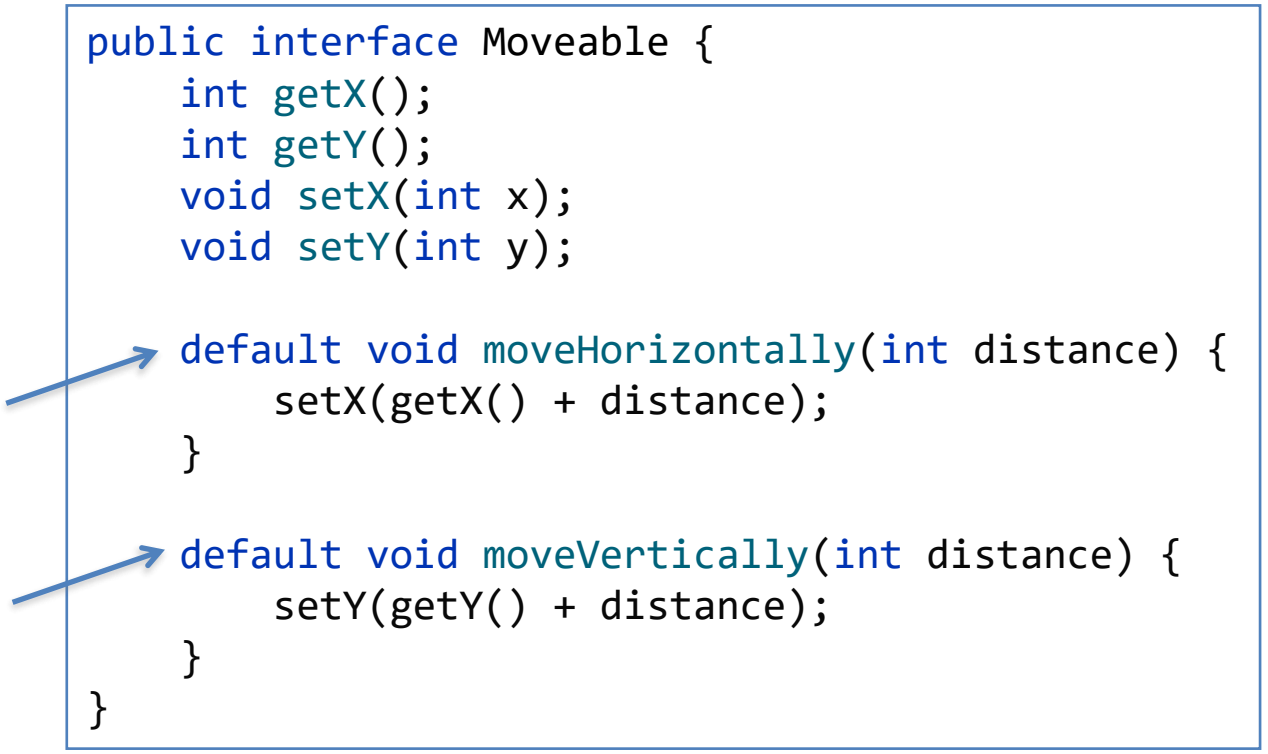
    @Override
    public void scale(double scaleFactor) {
        //...
    }

    @Override
    public int compareTo(Circle o) {
        //...
    }
}
```



Méthodes par défaut (Java 8+)

Il est possible, depuis Java 8, de définir, dans une interface, des méthodes contenant du code (méthodes par défaut).



```java
public interface Moveable {
    int getX();
    int getY();
    void setX(int x);
    void setY(int y);

    default void moveHorizontally(int distance) {
        setX(getX() + distance);
    }

    default void moveVertically(int distance) {
        setY(getY() + distance);
    }
}
```

Les classes qui implémentent une telle interface ont la liberté de redéfinir ou non ces méthodes par défaut.

Depuis Java 9, il est aussi possible d'y définir des méthodes privées utilisables dans les méthodes par défaut pour factoriser du code.

EPFC

Interfaces et collections `java.util`

Interface `Iterator<E>`

Un `Iterator<E>` est un objet permettant de parcourir les éléments `E` d'un autre objet.

Il définit les méthodes suivantes :

`boolean hasNext()`
: renvoie `true` s'il reste des éléments à parcourir

`E next()`
: passe à l'élément suivant et le renvoie.

`void remove()`
: Supprime le dernier élément retourné par l'itérateur
Cette opération est optionnelle car elle est implémentée via une méthode par défaut qui déclenche une `UnsupportedOperationException`.

Souvenez vous de SeqInt en PRM2…

Interface `Iterable<E>`

Un `Iterable<E>` est un objet qui peut fournir un `Iterator<E>`.
Il doit définir la méthode `Iterator<E> iterator()` qui renvoie l'`Iterator<E>` associé.

Ainsi, si `Calendar` est `Iterable<Date>`, on peut écrire :

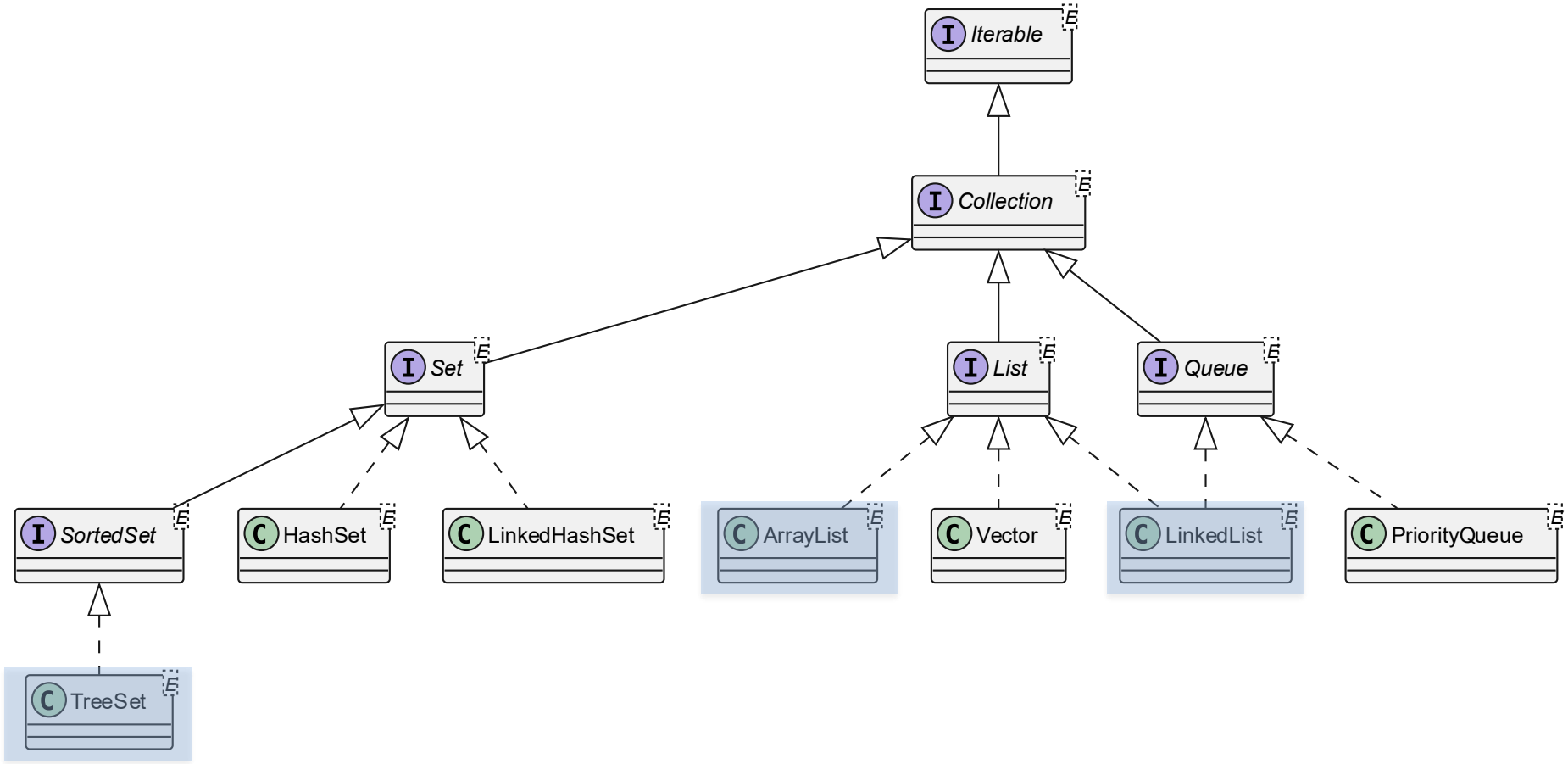
```
Calendar myCalendar = new Calendar();
Iterator<Date> it = myCalendar.iterator();

while(it.hasNext()) {
    Date d = it.next();
    System.out.println(d);
}
```

La boucle *foreach* est conçue pour parcourir de façon simple les itérables :

```
for(Date d : myCalendar) {
    System.out.println(d);
}
```

Les collections de `java.util`

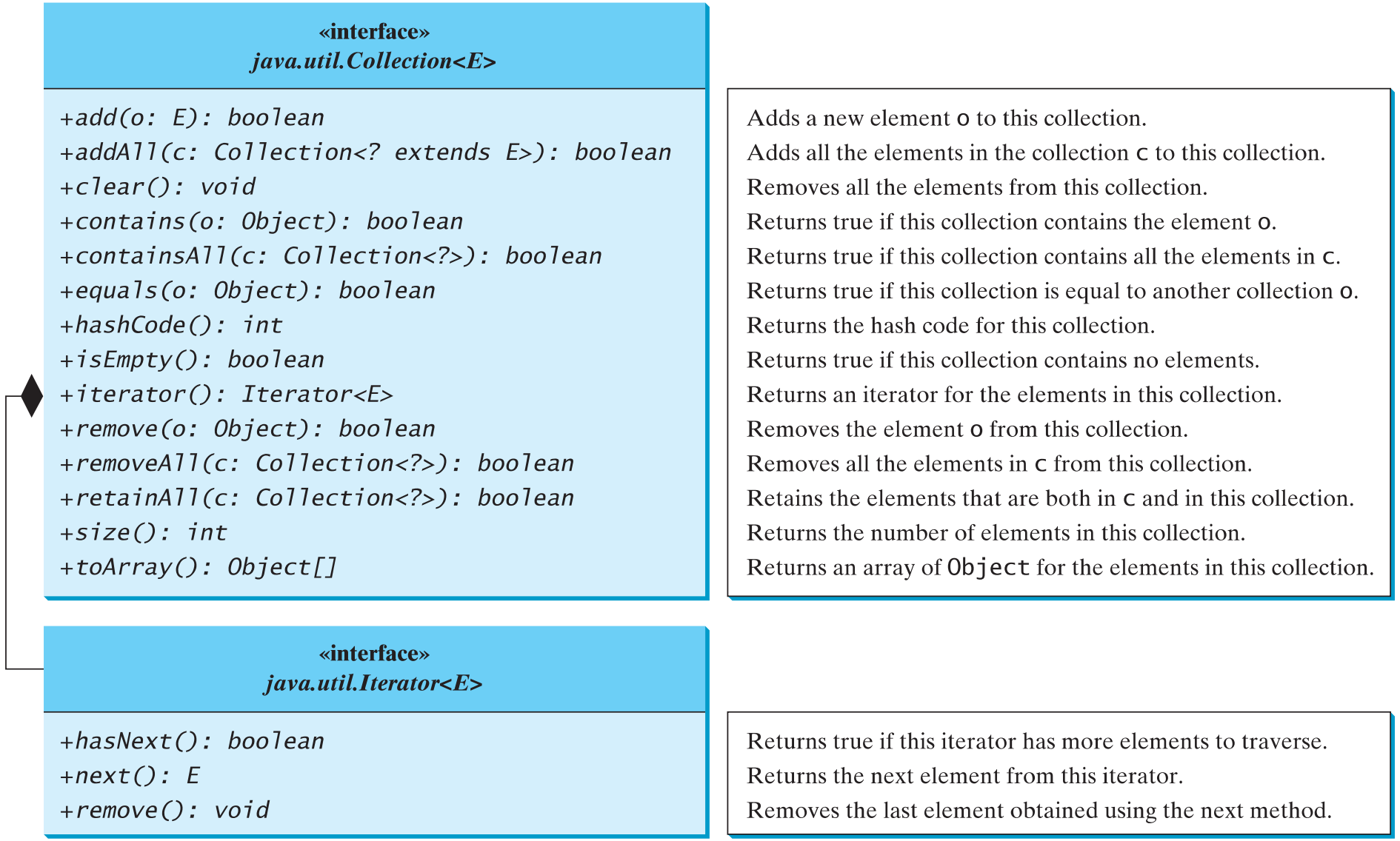


Dans cette leçon, nous parlerons des classes `TreeSet`, `ArrayList` et `LinkedList`.

Remarquez qu'une interface peut étendre (`extends`, trait continu) une autre interface

Interface Collection<E>

Source : Introduction to Java Programming, Y. Daniel Yang.



Interface `List<E>` et ses implémentations

EPFC

Une liste (`List<E>`) est une collection ordonnée d'éléments du même type.

> Ordonné (*ordered*) signifie que les éléments sont indicés et donc se trouvent à une certaine position.
> Cela ne signifie pas qu'ils sont triés (*sorted*).

Une `ArrayList<E>` est une liste implémentée à l'aide d'un tableau.

Une `LinkedList<E>` est implémentée à l'aide d'une liste liée.

Exemples :

```
List<Date> list1 = new LinkedList<>();
LinkedList<Date> list2 = new LinkedList<>();
```

Interface `Set<E>` et l'implémentation `TreeSet<E>`

EPFC

Un *set* (ensemble) est une collection non ordonnée qui ne contient pas de doublon.

Il utilise, entre autres, la méthode `equals()` pour déterminer si deux objets sont identiques.

Ces objets doivent être immuables ! (voir plus loin)

Un `TreeSet<E>` en est une implémentation qui garantit que les éléments sont rangés dans l'ordre croissant.

Il possède plusieurs constructeurs dont :

```
TreeSet<E>()
TreeSet<E>(Comparator<E> c)
```

Il existe d'autres implémentations de `Set<E>` que nous verrons plus tard.

Méthodes principales

Méthodes	List<E>	Set<E>
`boolean add(E elem)`	✔	✔
`boolean add(int index, E elem)`	✔	✖
`boolean contains(Object elem)`	✔	✔
`E get(int index)`	✔	✖
`Iterator<E> iterator()`	✔	✔
`boolean remove(int index)`	✔	✖
`boolean remove(Object elem)`	✔	✔
`int size()`	✔	✔

Retour sur la comparaison d'objets

Interface Comparable<E>

L'interface Comparable<E> permet d'indiquer à la librairie Java que les instances d'une classe sont comparables à des objets de type E et que donc la méthode compareTo(E o) y est implémentée.

```java
public class Person implements Comparable<Person> {
    //...
    @Override
    public int compareTo(Person p) {
        return -this.dateOfBirth.compareTo(p.dateOfBirth);
    }
}
```

La méthode statique Collections.sort(List<E>) trie une liste d'éléments comparables (qui implémentent donc l'interface Comparable<E>).

Interface Comparator<E>

Il est parfois utile de pouvoir comparer des objets E selon un critère donné, différent de celui implémenté dans `compareTo()`.

Pour cela, on peut définir un ou plusieurs `Comparator<E>`.

```
public class ComparatorPersonLastName implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getLastName().compareTo(p2.getLastName());
    }
}
```

```
List<Person> list = new ArrayList<>();
//...
Collections.sort(list, new ComparatorPersonLastName());
```

Depuis Java 8, l'interface `List<E>` possède une méthode par défaut `sort(Comparator<E>)` qui appelle `Collections.sort(List<E>, Comparator<E>)`. On peut donc appeler cette méthode directement sur une liste sans rien « casser ».

Résumé sur les interfaces

EPFC

Une interface définit un protocole de communication entre deux objets.

La déclaration d'une interface contient principalement des signatures de méthodes abstraites et peut aussi contenir des définitions de méthodes (méthodes par défaut, méthodes privées), des définitions de méthodes statiques et des définitions de constantes.

Une classe qui implémente une interface doit implémenter toutes les méthodes abstraites de l'interface. Cette classe peut également redéfinir une méthode par défaut héritée.

Le nom d'une interface peut être utilisé partout où un type peut être utilisé.

Source : http://docs.oracle.com/javase/tutorial/java/landl/summary-interface.html